hostrada4py - Benutzerhandbuch

- 1. Zweck und Funktionsumfang
- 2. Installation
- 3. Datenquellen, Variablen und Koordinaten
- 4. Grundkonzepte: Zeit, Provider und Cache
- 5. Schnellstart: Wetterdaten für einen Punkt
- 6. Mehrere Variablen an einem Punkt
- 7. Diffuse und direkte Solarstrahlung
- 8. Flächenauswertung mit Polygonen
- 9. Kartendarstellung von Flächendaten
- 10. Klimadaten entlang einer Route
- 11. Wetterdateien für Simulationsprogramme
- 12. Provider wechseln: DWD HOSTRADA und CERRA
- 14. Weiterverarbeitung und Qualitätssicherung
- 15. Fehlerbehebung
- 16. API-Kurzreferenz
- 17. Notebooks und weiterführende Beispiele
- 18. Lizenz und Quellen

Sprache: Deutsch

Dokumentationsstand: 8. August 2026

Bezug: `hostrada4py` 0.42.0, Zweig `main` des offiziellen GitHub-Repositories

Dieses Benutzerhandbuch beschreibt die praktische Arbeit mit der Python-Bibliothek `hostrada4py`. Es deckt den Zugriff auf DWD-HOSTRADA-Daten und den optionalen CERRA-Provider, Punkt-, Flächen- und Routenauswertungen, die Zerlegung der Globalstrahlung sowie den Export von Wetterdateien für Simulationsprogramme ab.

> Note
>
> Die Bibliothek und die zugrunde liegenden Datenquellen werden weiterentwickelt. Für reproduzierbare Projekte sollten Sie die verwendete `hostrada4py`-Version, den Provider, die Zeitbasis und die Cache-Strategie festhalten. Dieses Handbuch beschreibt den Repository-Stand vom 8. August 2026.

1. Zweck und Funktionsumfang

`hostrada4py` ist eine Open-Source-Python-Bibliothek der UdK Berlin, Fachgebiet Versorgungsplanung und Versorgungstechnik (VPT). Sie vereinfacht den Zugriff auf hochaufgelöste Wetterdaten und stellt Auswertungsfunktionen bereit, die direkt mit `pandas`, `xarray` und `GeoPandas` weiterverarbeitet werden können.

Typische Anwendungsfälle sind:

- stündliche Wetterzeitreihen für einen Standort,
- gleichzeitige Abfrage mehrerer meteorologischer Variablen,
- Ableitung von diffuser und direkter Solarstrahlung aus der Globalstrahlung,
- räumliche Auswertung über Stadt-, Regions- oder frei definierte Polygone,
- zeitliche Mittelwerte und räumliche Zusammenfassungen,
- interaktive Karten mit `folium`,
- Wetterwerte entlang zeitlich definierter Routen,
- Erzeugung von Wetterdateien für EnergyPlus, IDA ICE, Polysun, SimStadt und Modelica BuildingSystems,
- Wechsel zwischen DWD HOSTRADA und Copernicus CERRA über eine gemeinsame Provider-Schnittstelle.

Die öffentlichen Hilfsfunktionen arbeiten mit kanonischen Variablencodes. Dadurch kann ein großer Teil des Anwendungscodes unabhängig von der konkreten Datenquelle bleiben.

2. Installation

2.1 Voraussetzungen

Für `hostrada4py` 0.42.0 wird Python 3.10 oder neuer vorausgesetzt. Für die meisten Workflows benötigen Sie außerdem eine Internetverbindung, da Monatsdateien oder CERRA-Rohdaten bei Bedarf heruntergeladen werden.

Eine virtuelle Umgebung wird empfohlen:

```
python -m venv .venv

# Linux/macOS
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

2.2 Installation aus dem GitHub-Repository

Der im Projekt-README dokumentierte Entwicklungsweg ist eine editierbare Installation aus dem geklonten Repository:

```
git clone https://github.com/UdK-VPT/hostrada4py.git
cd hostrada4py
python -m pip install --upgrade pip
python -m pip install -e .
```

Prüfen Sie danach den Import und die Version:

```
import hostrada4py

print(hostrada4py.__version__)
# Erwartet für diesen Handbuchstand: 0.42.0
```

Zu den regulären Abhängigkeiten gehören unter anderem `numpy`, `pandas`, `xarray`, `netcdf4`, `pyproj`, `shapely`, `geopandas`, `folium`, `matplotlib`, `plotly`, `pvlib`, `fsspec` und `h5netcdf`.

2.3 Optionale CERRA-Abhängigkeiten

Für den Copernicus-CERRA-Provider sind zusätzliche Pakete erforderlich. Bei der aktuellen Paketdefinition können diese als optionales Extra installiert werden:

```
python -m pip install -e ".[cerra]"
```

Alternativ kann die im Repository enthaltene Datei `requirements-cerra.txt` verwendet werden:

```
python -m pip install -r requirements-cerra.txt
```

Dazu gehören insbesondere `cdsapi`, `cfgrib`, `eccodes` und `scipy`. Zusätzlich müssen gültige Zugangsdaten für den Copernicus Climate Data Store vorliegen. Details finden Sie in Abschnitt 12.

3. Datenquellen, Variablen und Koordinaten

3.1 DWD HOSTRADA

HOSTRADA ist ein hochaufgelöster stündlicher Rasterdatensatz des Deutschen Wetterdienstes für Deutschland. Der Datensatz deckt die Landflächen Deutschlands mit einer räumlichen Auflösung von etwa 1 km x 1 km ab und reicht bis 1995 zurück. Die Monatsdateien liegen als NetCDF im Koordinatenreferenzsystem EPSG:3034 vor. Zeitangaben in den Quelldaten werden in UTC geführt.

3.2 Copernicus CERRA

CERRA ist eine europäische regionale Reanalyse. Der `hostrada4py`-Provider normalisiert CERRA-Daten auf eine gemeinsame Schnittstelle und ein rectilineares EPSG:3034-Zielraster. Die aktuelle Implementierung verwendet standardmäßig ungefähr 5,5 km Rasterweite. Analysedaten liegen nativ dreistündlich vor und werden für die gemeinsame Schnittstelle stündlich normalisiert; für die Globalstrahlung werden stündliche Vorhersageakkumulationen genutzt. Der Provider unterstützt Daten ab September 1984. Ein HOSTRADA-äquivalentes UHI-Feld steht in CERRA nicht zur Verfügung.

3.3 Kanonische Variablencodes

Die folgenden Codes werden von `hostrada4py` verwendet. Die tatsächlich im zurückgegebenen DataFrame angegebene Einheit sollten Sie immer über die Spalte `unit` beziehungsweise über die Provider-Metadaten prüfen, da die Quellformate unterschiedliche Einheiten verwenden können.

`tas`
: Lufttemperatur in 2 m Höhe.

`tdew`
: Taupunkttemperatur in 2 m Höhe.

`uhi`
: Intensität der städtischen Wärmeinsel. Nur beim DWD-HOSTRADA-Provider verfügbar.

`sfcWind`
: Windgeschwindigkeit in 10 m Höhe.

`sfcWind_direction`
: Windrichtung in 10 m Höhe.

`rsds`
: Abwärts gerichtete kurzwellige Globalstrahlung an der Oberfläche.

`clt`
: Gesamtbewölkung.

`hurs`

Relative Luftfeuchte in 2 m Höhe.

mixr
Wasserdampf-Mischungsverhältnis in 2 m Höhe.

ps
Luftdruck auf Stations- beziehungsweise Oberflächenhöhe.

psl
Auf Meereshöhe reduzierter Luftdruck.

Die vom aktuell gewählten Provider unterstützten Variablen können programmgesteuert abgefragt werden:

```
from hostrada4py import hostrada as hs

print(hs.available_providers())
print(hs.supported_variables())
print(hs.provider_capabilities())
```

3.4 Koordinaten

An die öffentlichen Punkt-, Flächen- und Wetterdateifunktionen werden Längen- und Breitengrade in geographischen Koordinaten übergeben, also typischerweise `lon/lat` in EPSG:4326. Intern transformiert die Bibliothek auf das EPSG:3034-Raster der normalisierten Provider-Dateien.

Bei einer Punktabfrage wird die räumlich nächstgelegene Rasterzelle ausgewählt. Die Rückgabe enthält neben den Eingabekoordinaten je nach Datenquelle zusätzliche Metadaten wie `grid_x_epsg3034`, `grid_y_epsg3034`, `grid_lon` und `grid_lat`. So können Sie nachvollziehen, welche Rasterzelle tatsächlich benutzt wurde.

4. Grundkonzepte: Zeit, Provider und Cache

4.1 Zeitangaben bei Punkt- und Flächenabfragen

Punkt- und Flächenextraktionen arbeiten intern in UTC. Wenn Sie einen Zeitstempel ohne Zeitzoneninformation übergeben, interpretiert `hostrada4py` diesen bei den Punktfunktionen als UTC. Ein zeitzonenbehafteter Zeitstempel wird nach UTC konvertiert.

Für eindeutigen und reproduzierbaren Code ist eine explizite UTC-Angabe sinnvoll:

```
start = "2024-07-01T00:00:00Z"
end = "2024-07-02T23:00:00Z"
```

4.2 Zeitangaben bei Routen

Die Routen-API ist auf lokale Reisezeiten ausgelegt. Standardmäßig wird `Europe/Berlin` verwendet. Naive Zeitstempel in der Routentabelle werden in der angegebenen `timezone` lokalisiert; bereits zeitzonenbehaftete Werte werden in diese Zeitzone konvertiert. Bei mehrdeutigen oder nicht

existierenden lokalen Zeitpunkten während einer Sommerzeitumstellung wird absichtlich ein Fehler ausgelöst, damit keine stille Fehlinterpretation entsteht.

4.3 Provider

Der Standard-Provider ist `dwd`. Sie können einen Provider pro Funktionsaufruf, für einen Codeblock oder pro Prozess festlegen. Für reproduzierbare Skripte ist ein explizites `provider=` besonders transparent.

```
from hostrada4py import hostrada as hs
from hostrada4py.hostradaPoint import extract_values_for_point

# Explizit für einen Aufruf
df = extract_values_for_point(
    "tas", 13.405, 52.52,
    "2024-01-01T00:00:00Z", "2024-01-01T23:00:00Z",
    provider="dwd",
)

# Temporär für mehrere Aufrufe
with hs.use_provider("cerra"):
    print(hs.supported_variables())

# Prozessweiter Standard
hs.set_default_provider("dwd")
```

Alternativ kann der Prozessstandard über die Umgebungsvariable `HOSTRADA_PROVIDER` gesetzt werden.

4.4 Cache

Standardmäßig wird der Ordner `hostrada_cache` verwendet. DWD-HOSTRADA-Dateien werden monatsweise zwischengespeichert. Dadurch sind wiederholte Abfragen für bereits vorhandene Monate deutlich schneller und verursachen keine erneuten vollständigen Downloads.

Sie können einen eigenen Cache-Pfad übergeben:

```
df = extract_values_for_point(
    "tas", 13.405, 52.52,
    "2024-01-01", "2024-01-02",
    cache_dir="./cache/hostrada",
)
```

5. Schnellstart: Wetterdaten für einen Punkt

Die zentrale Funktion ist `extract_values_for_point`.

```python
from hostrada4py.hostradaPoint import extract_values_for_point

df = extract_values_for_point(
    var="tas",
    lon=13.4050,
    lat=52.5200,
    start="2024-07-01T00:00:00Z",
    end="2024-07-02T23:00:00Z",
    provider="dwd",
    cache_strategy="full",
    verbose=True,
)

print(df.head())
print(df.columns)
```

Die Rückgabe ist ein `pandas.DataFrame`. Mindestens enthalten sind die Spalten `time` und die abgefragte Variable, hier also `tas`. Typischerweise kommen Metadaten zur gewählten Rasterzelle sowie `unit` und `variable_description` hinzu.

Wichtige Hinweise:

- `end` darf nicht vor `start` liegen.
- Die Eingabekoordinate muss ein gültiger Längen-/Breitengrad sein.
- Außerhalb der räumlichen Abdeckung eines Providers kann die nächstgelegene Zelle fachlich ungeeignet sein oder der Provider kann die Anfrage ablehnen.
- Prüfen Sie `unit`, bevor Sie numerische Schwellenwerte oder physikalische Formeln anwenden.
- Für Analysen über viele Monate lohnt es sich, den Cache dauerhaft zu behalten.

Ein einfaches Diagramm:

```python
import matplotlib.pyplot as plt

ax = df.plot(x="time", y="tas", figsize=(10, 4), legend=False)
ax.set_title("Lufttemperatur - Berlin")
ax.set_xlabel("Zeit")
ax.set_ylabel(df["unit"].dropna().iloc[0] if "unit" in df else "tas")
plt.tight_layout()
plt.show()
```

6. Mehrere Variablen an einem Punkt

Für mehrere Variablen nutzen Sie `extract_multiple_values_for_point`. Die Einzelzeitreihen werden auf der Zeitachse zu einem breiten DataFrame kombiniert.

```python
from hostrada4py.hostradaPoint import extract_multiple_values_for_point

weather = extract_multiple_values_for_point(
    variables=["tas", "hurs", "sfcWind", "rsds"],
    lon=13.4050,
    lat=52.5200,
    start="2024-07-01T00:00:00Z",
    end="2024-07-07T23:00:00Z",
    provider="dwd",
    cache_strategy="full",
)

print(weather.head())
print(weather.index)
```

Die kombinierte Rückgabe verwendet einen `DatetimeIndex` und enthält die Variablen als Spalten. Standortmetadaten der ersten geeigneten Einzelabfrage werden beibehalten.

Ein praktischer Export:

```python
weather.reset_index().to_csv(
    "berlin_weather.csv",
    index=False,
    encoding="utf-8",
)
```

7. Diffuse und direkte Solarstrahlung

HOSTRADA und CERRA liefern Globalstrahlung, aber nicht direkt die diffuse Horizontalstrahlung (DHI). `hostrada4py` kann die Globalstrahlung mit `pvlib` und dem Modell `erbs_driesse` zerlegen.

7.1 Komfortfunktion

```python
from hostrada4py import extract_diffuse_radiation_for_point

solar = extract_diffuse_radiation_for_point(
    lon=13.405,
    lat=52.52,
    start="2024-06-01T00:00:00Z",
    end="2024-06-03T23:00:00Z",
    tz="Europe/Berlin",
    provider="dwd",
    apply_weather_correction=False,
)

print(solar[["dhi", "dni", "kd"]].head())
```

Die wichtigsten abgeleiteten Größen sind:

`dhi`
: Diffuse Horizontalstrahlung.

`dni`
: Direkte Normalstrahlung.

`kd`

Diffuser Anteil der Globalstrahlung zwischen 0 und 1.

Zusätzlich werden Solarstandsinformationen wie `solar_zenith_deg`, `solar_elevation_deg` und `solar_azimuth_deg` ergänzt.

7.2 Optionale Wetterkorrektur

Mit `apply_weather_correction=True` wird die robuste `erbs_driesse`-Basis um eine begrenzte, konservative Wetterkorrektur erweitert. Abhängig vom Provider werden zusätzlich verfügbare Variablen wie Bewölkung, relative Luftfeuchte oder UHI berücksichtigt. Nicht verfügbare Korrekturvariablen werden nicht erzwungen.

```
solar_corrected = extract_diffuse_radiation_for_point(
    lon=13.405,
    lat=52.52,
    start="2024-06-01T00:00:00Z",
    end="2024-06-03T23:00:00Z",
    tz="Europe/Berlin",
    apply_weather_correction=True,
)
```

Für wissenschaftliche oder normative Anwendungen sollten Sie dokumentieren, ob die optionale Korrektur aktiviert wurde und die abgeleiteten Strahlungsgrößen gegen die Anforderungen Ihres Modells prüfen.

7.3 Direkte Nutzung von HostradaDiffuse

Wenn Sie bereits einen eigenen DataFrame mit `rsds` haben, können Sie die Klasse `HostradaDiffuse` direkt verwenden. Der Index muss ein `pandas.DatetimeIndex` sein.

```
from hostrada4py import HostradaDiffuse

model = HostradaDiffuse(
    latitude=52.52,
    longitude=13.405,
    altitude=34.0,
    tz="Europe/Berlin",
)

result = model.estimate(weather, apply_weather_correction=False)
```

8. Flächenauswertung mit Polygonen

Mit `hostradaArea` können Sie alle Rasterzellen innerhalb eines Polygons extrahieren. Polygone werden als Folge von `(lon, lat)`-Paaren angegeben.

8.1 Eigenes Polygon

```python
from hostrada4py.hostradaArea import extract_values_for_polygon

polygon = [
    (13.30, 52.45),
    (13.55, 52.45),
    (13.55, 52.60),
    (13.30, 52.60),
    (13.30, 52.45),
]

area = extract_values_for_polygon(
    var="tas",
    polygon_lonlat=polygon,
    start_utc="2024-07-01T12:00:00Z",
    end_utc="2024-07-01T15:00:00Z",
    selection_mode="within",
    return_geodataframe=True,
    provider="dwd",
    cache_strategy="subset",
    subset_margin_cells=1,
)

print(area.head())
print(area.crs)
```

Bei `return_geodataframe=True` erhalten Sie ein `GeoDataFrame` mit EPSG:3034. Typische Spalten sind `time`, `grid_y_epsg3034`, `grid_x_epsg3034`, die Wertespalte, `geometry`, `selection_mode`, `grid_lon` und `grid_lat`.

8.2 Vordefinierte Städte

Das Modul `hostradaCities` enthält grobe Rechteckpolygone für die 50 größten Städte Deutschlands sowie Speyer und Korbach. Diese Polygone sind bewusst einfache räumliche Auswahlbereiche und keine amtlichen Verwaltungsgrenzen.

```python
from hostrada4py.hostradaCities import CITY_POLYGONS
from hostrada4py.hostradaArea import extract_values_for_polygon

berlin = CITY_POLYGONS["Berlin"]

berlin_tas = extract_values_for_polygon(
    "tas",
    berlin,
    "2024-07-01T12:00:00Z",
    "2024-07-01T12:00:00Z",
)
```

Zusätzlich existieren aus Kompatibilitätsgründen Variablennamen wie `berlin_polygon`.

8.3 Vordefinierte Regionen

`hostradaRegions` enthält einfache Auswahlpolygone für Deutschland und mehrere Bundesländer beziehungsweise Regionen.

```python
from hostrada4py.hostradaRegions import REGIONS_POLYGONS

germany = REGIONS_POLYGONS["Germany"]
brandenburg = REGIONS_POLYGONS["Brandenburg"]
```

Auch diese Geometrien sind Auswahlboxen und nicht als amtliche Grenzgeometrien zu interpretieren.

8.4 Zeitlicher Mittelwert je Rasterzelle

`extract_mean_values_for_polygon` reduziert einen Zeitraum auf eine Zeile pro Rasterzelle. Optional werden zusätzliche Statistiken erzeugt.

```python
from hostrada4py.hostradaArea import extract_mean_values_for_polygon

mean_area = extract_mean_values_for_polygon(
    var="tas",
    polygon_lonlat=berlin,
    start_utc="2024-07-01T00:00:00Z",
    end_utc="2024-07-07T23:00:00Z",
    include_statistics=True,
)

print(mean_area.head())
```

8.5 Räumliche Zusammenfassung je Zeitpunkt

Wenn Sie bereits die vollständige Flächenzeitreihe besitzen, berechnet `summarize_values_period` pro Zeitpunkt Mittelwert, Minimum, Maximum und die Anzahl ausgewerteter Rasterzellen.

```python
from hostrada4py.hostradaArea import summarize_values_period

summary = summarize_values_period(area, "tas")
print(summary.head())
```

9. Kartendarstellung von Flächendaten

`hostradaArea` kann `folium`-Karten für einen einzelnen Zeitpunkt oder einen zeitlich aggregierten Datensatz erzeugen.

9.1 Karte eines Zeitpunkts

```python
from hostrada4py.hostradaArea import make_leaflet_map_timepoint

m = make_leaflet_map_timepoint(
    area,
    var="tas",
    time_utc="2024-07-01T12:00:00Z",
    show_cell_values=True,
    decimals=1,
    title="Berlin - Lufttemperatur",
    save_html="berlin_tas_2024-07-01_12UTC.html",
)
```

In einem Jupyter Notebook wird das zurückgegebene `folium.Map`-Objekt direkt angezeigt. In einem Skript ist `save_html` praktisch, um die interaktive Karte im Browser zu öffnen.

9.2 Karte eines Zeitraums

```python
from hostrada4py.hostradaArea import make_leaflet_map_timeperiod

m_mean = make_leaflet_map_timeperiod(
    mean_area,
    var="tas",
    show_cell_values=False,
    title="Berlin - mittlere Lufttemperatur",
    save_html="berlin_tas_mean.html",
)
```

Mit `vmin` und `vmax` können Sie eine feste Farbskala für vergleichbare Karten definieren. `reverse_colormap=True` kehrt die Farbskala um.

10. Klimadaten entlang einer Route

Die Routenfunktion verknüpft Position und Zeitpunkt jedes Routenpunkts mit den Wetterdaten. Sie akzeptiert einen `pandas.DataFrame` oder eine CSV-Datei.

10.1 Eingabespalten

Erkannt werden folgende Spaltennamen, unabhängig von Groß-/Kleinschreibung:

- Zeit: `timestamp`, `time`, `datetime` oder `date_time`
- Länge: `longitude`, `lon`, `lng` oder `x`
- Breite: `latitude`, `lat` oder `y`

10.2 Beispiel mit DataFrame

```python
import pandas as pd
from hostrada4py.hostradaRoute import calculate_route_climate

route = pd.DataFrame(
    {
        "timestamp": [
            "2024-07-01 08:00:00",
            "2024-07-01 08:30:00",
            "2024-07-01 09:00:00",
        ],
        "longitude": [13.4050, 13.20, 12.95],
        "latitude": [52.5200, 52.48, 52.40],
    }
)

route_weather = calculate_route_climate(
    route,
    variables=["tas", "hurs", "sfcWind", "rsds"],
    timezone="Europe/Berlin",
    interpolation="linear",
    provider="dwd",
    output_csv="route_weather.csv",
)

print(route_weather.head())
```

Die Standardinterpolation ist `linear`. Alternativ ist `nearest` verfügbar. Bei Windrichtungen berücksichtigt die lineare Interpolation den zyklischen Übergang bei 0/360 Grad.

Die Ausgabe enthält für jede Variable neben dem aufbereiteten Ergebnis zusätzliche Provenienzfelder, zum Beispiel Quellwert, Quelleneinheit, die beiden verwendeten UTC-Zeitpunkte, den Interpolationsanteil und die Rasterkoordinate.

10.3 Alle unterstützten Variablen

Mit `variables="all"` werden alle Variablen ausgewählt, die sowohl die Routenimplementierung kennt als auch der aktive Provider unterstützt.

```python
from hostrada4py.hostradaRoute import available_variables

print(available_variables(provider="dwd"))
```

10.4 Kommandozeile

Das Routenmodul enthält auch einen kleinen CLI-Einstieg:

```bash
python -m hostrada4py.hostradaRoute route.csv \
    -o route_weather.csv \
    -v tas hurs rsds \
    --timezone Europe/Berlin \
    --interpolation linear \
    --cache-strategy full \
    --provider dwd
```

Für Routen ist `full` eine sinnvolle Standardstrategie, weil viele Punkte eines Monats dieselben Monatsdateien wiederverwenden können.

11. Wetterdateien für Simulationsprogramme

Das Repository enthält Exportfunktionen für mehrere Simulationsumgebungen. Alle arbeiten nach demselben Grundmuster: Standort und Zeitraum angeben, optional Provider, Zeitzone, Höhe, Cache-Strategie und Strahlungskorrektur setzen, Zieldatei schreiben.

11.1 EnergyPlus EPW

```
from hostrada4py.hostrada_EnergyPlus_Weather import (
    create_energyplus_weather_file,
)

path = create_energyplus_weather_file(
    lon=13.405,
    lat=52.52,
    start="2024-01-01T00:00:00Z",
    end="2024-12-31T23:00:00Z",
    output_file="Berlin_2024.epw",
    location_name="Berlin",
    country="DEU",
    tz="Europe/Berlin",
    altitude=34.0,
    provider="dwd",
    cache_strategy="subset",
    subset_margin_cells=1,
)

print(path)
```

11.2 IDA ICE

```
from hostrada4py.hostrada_IDA_ICE_Weather import (
    create_ida_ice_weather_file,
)

create_ida_ice_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024.prn",
    tz="Europe/Berlin",
    provider="dwd",
)
```

11.3 Polysun

```
from hostrada4py.hostrada_Polysun_Weather import (
    create_polysun_weather_file,
)

create_polysun_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024_Polysun.csv",
    tz="Europe/Berlin",
)
```

11.4 SimStadt

```
from hostrada4py.hostrada_SimStadt_Weather import (
    create_simstadt_weather_file,
)

create_simstadt_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024.tmy3",
    tz="Europe/Berlin",
)
```

11.5 Modelica BuildingSystems

```
from hostrada4py.hostrada_BuildingSystems_Weather import (
    create_buildingsystems_weather_file,
)

create_buildingsystems_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024.mos",
    tz="UTC",
)
```

> Warning
>
> Prüfen Sie vor einer Simulation immer, ob Zeitraum, Zeitzone, Schaltjahr, Höhenangabe, Strahlungsgrößen und Einheiten den Anforderungen des Zielprogramms entsprechen. Ein erfolgreich erzeugtes Dateiformat ist noch keine fachliche Validierung des Simulationsdatensatzes.

12. Provider wechseln: DWD HOSTRADA und CERRA

12.1 Verfügbare Provider und Fähigkeiten

```python
from hostrada4py import hostrada as hs

for name in hs.available_providers():
    cap = hs.provider_capabilities(name)
    print(name)
    print("  Variablen:", sorted(cap.variables))
    print("  Zeitauflösung:", cap.temporal_resolution)
    print("  Raster [m]:", cap.spatial_resolution_m)
    print("  CRS:", cap.crs)
    print("  Start:", cap.start)
```

Aktuell werden `dwd` und `cerra` registriert. Für DWD existieren zusätzlich die Aliase `hostrada` und `dwd_hostrada`; für CERRA `copernicus_cerra`.

12.2 CERRA-Zugang konfigurieren

Für CERRA benötigen Sie ein Konto und API-Zugangsdaten des Copernicus Climate Data Store. Die Bibliothek sucht nach einer optionalen Datei `hostrada4py/cerra_config.json`. Fehlen dort Werte oder die Datei selbst, greift `cdsapi` auf seine übliche Konfiguration wie `~/.cdsapirc` und unterstützte Umgebungsvariablen zurück.

Ein mögliches lokales JSON-Schema ist:

```json
{
  "url": "<CDS-API-URL>",
  "key": "<DEIN-API-SCHLUESSEL>"
}
```

Speichern Sie echte Zugangsdaten nicht in einem öffentlichen Git-Repository.

12.3 CERRA verwenden

```python
from hostrada4py.hostradaPoint import extract_values_for_point

cerra = extract_values_for_point(
    "tas",
    lon=2.3522,
    lat=48.8566,
    start="2024-01-01T00:00:00Z",
    end="2024-01-02T23:00:00Z",
    provider="cerra",
)

print(cerra.head())
```

Wesentliche Unterschiede zum DWD-Provider:

- CERRA deckt Europa statt nur Deutschland ab.
- Die normalisierte Rasterweite ist gröber als HOSTRADA.
- CERRA beginnt in der aktuellen Implementierung im September 1984.
- UHI ist nicht verfügbar.
- Der native CERRA-Datensatz ist für Analysen überwiegend dreistündlich; die Provider-Schicht erzeugt eine stündliche normalisierte Schnittstelle.
- Der CDS-Abruf kann große Rohdateien übertragen. Der aktuelle CERRA-Prozess bietet für diesen Datensatz keine echte geographische Teilabfrage; die Bibliothek schneidet daher nach dem Abruf lokal auf den Zielbereich zu.

13. Cache- und Downloadstrategien ----------

Die DWD-HOSTRADA-Seite stellt statische Monatsdateien bereit. `hostrada4py` bietet mehrere Strategien, um Downloadmenge und lokalen Speicherbedarf auszubalancieren.

`full`
: Lädt die vollständige Monatsdatei und behält sie im Cache. Dies ist der robuste Standard und besonders sinnvoll, wenn viele unterschiedliche Punkte oder Flächen desselben Monats verwendet werden.

`subset`
: Verwendet bei Bedarf die vollständige Datei, erzeugt daraus einen kleineren räumlich/zeitlich zugeschnittenen NetCDF-Cache und verwendet diesen bei späteren passenden Anfragen wieder.

`http_range`
: Versucht bestmöglich einen Zugriff über HTTP-Bytebereiche beziehungsweise chunk-basiertes Lesen. Ob dadurch ein echter Vorteil entsteht, hängt vom Server, Dateiformat und lokalen optionalen Paketen ab.

`auto`
: Versucht zunächst `http_range` und fällt anschließend auf `subset` zurück.

Beispiel:

```
df = extract_values_for_point(
    "tas",
    lon=13.405,
    lat=52.52,
    start="2024-01-01T00:00:00Z",
    end="2024-01-31T23:00:00Z",
    cache_strategy="subset",
    subset_margin_cells=1,
)
```

Die wichtigsten Umgebungsvariablen sind:

`HOSTRADA_NETCDF_SUBSET_MODE`
: Prozessweiter Standard für die DWD-Cache-Strategie, zum Beispiel `subset`.

`HOSTRADA_NETCDF_SUBSET_MARGIN_CELLS`
: Zusätzlicher Rand in Rasterzellen für Subsets.

`HOSTRADA_NETCDF_SUBSET_FALLBACK`
: Steuert, ob bei fehlgeschlagenem `http_range` auf die robuste Vollversion zurückgefallen wird. Mit `0` kann der Fallback deaktiviert werden.

HOSTRADA_DROP_FULL_AFTER_SUBSET
: Mit 1 kann die vollständige Monatsdatei nach erfolgreicher Erstellung eines Subsets gelöscht werden, um Speicherplatz zu sparen.

Beispiel in Python vor dem Import beziehungsweise vor dem ersten Abruf:

```
import os

os.environ["HOSTRADA_NETCDF_SUBSET_MODE"] = "subset"
os.environ["HOSTRADA_NETCDF_SUBSET_MARGIN_CELLS"] = "1"
```

Empfehlungen:

- Ein einzelner Standort über viele Monate: `subset` kann lokalen Speicher sparen.
- Viele Orte/Flächen im selben Monat: `full` ist häufig effizienter.
- Routen mit vielen Punkten: `full` ist der bewusst gewählte Standard der Routen-API.
- `http_range`: als Optimierung betrachten, nicht als Garantie für serverseitiges NetCDF-Subsetting.

14. Weiterverarbeitung und Qualitätssicherung

14.1 Daten immer inspizieren

Bevor Sie Ergebnisse aggregieren oder exportieren, prüfen Sie:

```
print(df.head())
print(df.tail())
print(df.dtypes)
print(df.isna().sum())
print(df["time"].min(), df["time"].max())
```

Bei Punktdaten sollten Sie zusätzlich die tatsächlich gewählte Rasterzelle und die Einheit dokumentieren.

14.2 Zeitachse kontrollieren

Bei lokalen Auswertungen empfiehlt sich eine explizite Konvertierung:

```
import pandas as pd

local = df.copy()
local["time"] = (
    pd.to_datetime(local["time"], utc=True)
    .dt.tz_convert("Europe/Berlin")
)
```

So bleiben Sommerzeitwechsel nachvollziehbar.

14.3 Reproduzierbare Metadaten speichern

Für wissenschaftliche oder langfristige Projekte sollten mindestens folgende Informationen zusammen mit Ergebnissen abgelegt werden:

- `hostrada4py`-Version,
- Providername,
- Variablencode und zurückgegebene Einheit,
- Koordinate oder Polygon,
- Zeitraum und Zeitzone,
- Cache-Strategie,
- bei Strahlungszerlegung: Methode und Wetterkorrektur,
- Datum der Datenabfrage, da die Quelldatensätze monatlich erweitert werden.

Beispiel:

```python
import hostrada4py
from hostrada4py import hostrada as hs

metadata = {
    "hostrada4py_version": hostrada4py.__version__,
    "provider": hs.get_provider_name(),
    "query_timezone": "UTC",
    "cache_strategy": "full",
}
print(metadata)
```

15. Fehlerbehebung

`ModuleNotFoundError: hostrada4py`
: Prüfen Sie, ob die richtige virtuelle Umgebung aktiv ist und ob `python -m pip install -e .` im Repository ausgeführt wurde.

CERRA: `ImportError` zu `cdsapi`, `cfgrib` oder `eccodes`
: Installieren Sie die optionalen CERRA-Abhängigkeiten. Bei `eccodes` können je nach Betriebssystem zusätzlich native Bibliotheken oder Systempakete erforderlich sein.

CERRA: Authentifizierungs- oder CDS-Fehler
: Prüfen Sie Konto, akzeptierte Dataset-Nutzungsbedingungen und API-Konfiguration. Testen Sie zuerst eine kleine, kurze Anfrage.

`Unknown weather provider`
: Prüfen Sie `hs.available_providers()` und verwenden Sie zum Beispiel `dwd` oder `cerra`.

`Provider ... does not provide: uhi`
: UHI ist ein HOSTRADA-spezifisches Feld und wird vom CERRA-Provider bewusst nicht angeboten.

`end must not be earlier than start`
: Korrigieren Sie die Zeitgrenzen. Verwenden Sie nach Möglichkeit ISO-8601-Werte mit explizitem `Z` für UTC.

Leere Flächenausgabe oder keine geeigneten Zellen

Prüfen Sie Reihenfolge und Lage der `(lon, lat)`-Punkte, den Provider und die räumliche Abdeckung. Visualisieren Sie das Polygon bei Bedarf separat.

Sehr große Downloads oder wachsender Cache
Wählen Sie `subset` für punktbezogene Langzeitauswertungen oder löschen Sie nicht mehr benötigte Cache-Verzeichnisse bewusst. `HOSTRADA_DROP_FULL_AFTER_SUBSET=1` kann Speicher sparen, erhöht aber spätere Downloadkosten für andere Ausschnitte.

Route schlägt am Sommerzeitwechsel fehl
Die Routen-API wirft bei mehrdeutigen oder nicht existierenden lokalen Zeiten absichtlich einen Fehler. Liefern Sie für solche Punkte eindeutig zeitzonenbehaftete Zeitstempel mit UTC-Offset oder verwenden Sie UTC als Routenzeitzone.

Netzwerkfehler beim DWD-Abruf
Wiederholen Sie die Abfrage später und behalten Sie bereits vollständig geladene Monatsdateien im Cache. Bei experimentellem `http_range` kann `auto` oder `full` robuster sein.

16. API-Kurzreferenz

Punkt

```
extract_values_for_point(
    var, lon, lat, start, end,
    cache_dir="hostrada_cache",
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
) -> pandas.DataFrame

extract_multiple_values_for_point(
    variables, lon, lat, start, end,
    cache_dir="hostrada_cache",
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
) -> pandas.DataFrame
```

Historische Aliase: `extract_values` und `extract_multiple_variables_for_point`.

Diffuse Strahlung

```
extract_diffuse_radiation_for_point(
    lon, lat, start, end,
    tz="UTC",
    apply_weather_correction=False,
    cache_dir="hostrada_cache",
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
) -> pandas.DataFrame
```

Fläche

```
extract_values_for_polygon(
    var,
    polygon_lonlat,
    start_utc,
    end_utc,
    cache_dir="hostrada_cache",
    selection_mode="within",
    return_geodataframe=True,
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
)

extract_mean_values_for_polygon(
    var,
    polygon_lonlat,
    start_utc,
    end_utc,
    cache_dir="hostrada_cache",
    selection_mode="within",
    return_geodataframe=True,
    mean_column=None,
    include_statistics=True,
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
)

summarize_values_period(gdf_or_df, var) -> pandas.DataFrame
```

Karten

```
make_leaflet_map_timepoint(gdf_or_df, var, time_utc, ..., save_html=None)
make_leaflet_map_timeperiod(gdf_or_df, var, ..., save_html=None)
```

Route

```
calculate_route_climate(
    route,
    variables="all",
    output_csv=None,
    timezone="Europe/Berlin",
    interpolation="linear",
    cache_dir="hostrada_cache",
    cache_strategy="full",
    subset_margin_cells=0,
    continue_on_error=False,
    progress_callback=None,
    provider=None,
)
```

Aliase: `calculate_hostrada_values_along_route` und `extract_values_for_route`.

Provider

```
hostrada.available_providers()
hostrada.get_provider(provider=None)
hostrada.get_provider_name(provider=None)
hostrada.supported_variables(provider=None)
hostrada.provider_capabilities(provider=None)
hostrada.set_default_provider(provider)
hostrada.use_provider(provider)
```

Wetterdateien

```
create_energyplus_weather_file(..., output_file="HOSTRADA_EnergyPlus.epw", ...)
create_ida_ice_weather_file(..., output_file="HOSTRADA_IDA_ICE.prn", ...)
create_polysun_weather_file(..., output_file="HOSTRADA_Polysun.csv", ...)
create_simstadt_weather_file(..., output_file="HOSTRADA_SimStadt.tmy3", ...)
create_buildingsystems_weather_file(..., output_file="HOSTRADA_BuildingSystems.mos", ...)
```

17. Notebooks und weiterführende Beispiele

Im Repository liegen Jupyter-Notebooks, die die wichtigsten Anwendungsfälle illustrieren:

- `hostradaPoint.ipynb` - Zeitreihen für einen Standort
- `hostradaPoint-diffRad.ipynb` - diffuse Strahlung
- `hostradaRoute.ipynb` - Werte entlang einer Route
- `hostradaArea.ipynb` - zweidimensionale Wetterfelder
- `hostradaAreaMean.ipynb` - zeitliche Mittelwerte von Wetterfeldern
- `hostradaHeatPeriods.ipynb` - intensive Hitzeperioden
- `hostradaHeatingDegreeDays.ipynb` - Heizgradtage
- `hostradaGenerateWeatherFiles.ipynb` - Wetterdateien für Simulationsprogramme

Die Notebooks sind besonders nützlich, um Visualisierung, Eingabeformate und komplette Analyseketten zu sehen. Für produktive Skripte sollten Sie anschließend die in diesem Handbuch beschriebenen Funktionen direkt importieren und Provider, Zeitbasis sowie Cache explizit konfigurieren.

18. Lizenz und Quellen

Die Bibliothek `hostrada4py` wird im Repository unter der MIT-Lizenz bereitgestellt. Für die meteorologischen Quelldaten gelten die jeweiligen Bedingungen und Attributionsanforderungen von DWD beziehungsweise Copernicus. Prüfen Sie diese vor Veröffentlichung oder Weitergabe abgeleiteter Datensätze.

Wichtige Quellen für dieses Handbuch:

- Offizielles Repository: https://github.com/UdK-VPT/hostrada4py
- README: https://github.com/UdK-VPT/hostrada4py/blob/main/README.md
- Paketdefinition: https://github.com/UdK-VPT/hostrada4py/blob/main/pyproject.toml
- DWD HOSTRADA Open Data: https://opendata.dwd.de/climate_environment/CDC/grids_germany/hourly/hostrada/
- Copernicus CERRA: https://climate.copernicus.eu/copernicus-regional-reanalysis-europe-cerra
- Copernicus Climate Data Store: https://cds.climate.copernicus.eu/

Ende des Benutzerhandbuchs.